Laboratorio 2

Diego Stefanini — prof. Patrizio Dazzi, Luca Ferrucci, a.a. 2026-27

Indice

1 L'ambiente di lavoro UNIX 3
1.1 Il terminale e il filesystem 3
1.2 Lavorare con i file 4
1.3 Flussi: redirezione e pipe 6
2 Il linguaggio C 8
2.1 Dal sorgente all'eseguibile 8
2.2 Com'è fatto un programma C 10
2.3 Tipi e costanti 12
2.4 Operatori e controllo del flusso 15
2.5 Array e tipi custom 16
2.6 Funzioni e programmi su più file 19
3 Puntatori e memoria 23
3.1 Puntatori 23
3.2 Array e puntatori 24
3.3 Allocazione statica e dinamica 25
3.4 Array di puntatori e matrici dinamiche 26
3.5 Stringhe 28
3.6 Puntatori a funzioni 31
3.7 Trovare gli errori 31
4 Strutture dati 35
4.1 Puntatori a struct 35
4.2 Liste concatenate 36
4.3 Alberi binari di ricerca 38
Riepilogo 41

1 L'ambiente di lavoro UNIX

Cosa devi saper fare

- muoverti nel filesystem con percorsi assoluti e relativi (`pwd` , `ls` , `cd` , `.` , `..` , `~`);
- creare, copiare, spostare e cancellare file e directory, e dire quali timestamp cambiano;
- prevedere cosa fanno wildcard, redirezioni (`>` , `>>` , `<` , `2>`) e pipe `|` .

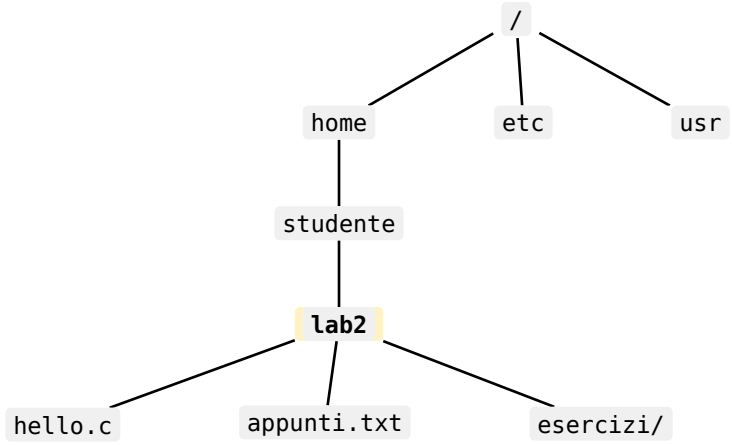
1.1 Il terminale e il filesystem

Si lavora dal **terminale**: una finestra in cui gira la **shell** (bash, zsh), il programma che legge i comandi e li esegue.

```
$ pwd
/home/studente/lab2
$ ls
appunti.txt  esercizi  hello.c
```

Il **prompt** (`$`) dice che la shell aspetta un comando. Non fa parte del comando e non è sempre `$` .

I comandi agiscono sul **filesystem**, un albero di directory e file:



Parte da `/` , la **root**: ogni file o directory si raggiunge con un **percorso**, in cui `/` separa un livello dal successivo.

File e directory sono oggetti diversi, ma nella stessa directory non possono avere lo stesso nome.

In giallo la **directory corrente**.

- `.` = la directory corrente
- `..` = la directory padre

Comando	Descrizione
`pwd`	*print working directory*: stampa in quale cartella sei
`ls`	elenca il contenuto della directory corrente
`ls -l`	elenco in formato lungo (permessi, proprietario, dimensione, data)
`cd`	*change directory*: navigo in una directory
`mkdir`	*make directory*: crea una directory
`echo $?`	restituisce il valore di ritorno dell'ultimo comando eseguito

Per indicare un file nell'albero si usa un **percorso**, assoluto o relativo:

Relativo: parte dalla directory corrente (es. `esercizi/es1.c`).

```
$ cd esercizi
$ pwd
/home/studente/lab2/esercizi
```

Assoluto: parte dalla root `/` (es. `/home/studente/lab2`).

```
$ cd /home/studente
$ pwd
/home/studente
```

```
$ pwd
/home/studente/lab2
$ cd ..          # vado nel padre
$ pwd
/home/studente
```

`~` = la **home** dell'utente corrente. `cd` senza argomenti porta di solito lì.

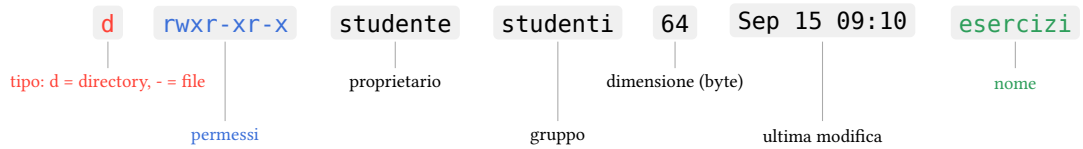
```
$ cd ~
$ pwd
/home/studente
```

Un nome che inizia con `.` è **nascosto**: `ls` non lo mostra, `ls -a` sì.

```
$ ls
appunti.txt  hello.c
$ ls -a
.  ..  .bashrc  appunti.txt  hello.c
```

> «Nascosto» è solo una **convenzione sul nome**, non un tipo speciale di file.

Con `ls -l` si vedono le informazioni su ogni file. Ecco come leggerle:



Ogni comando ha la stessa forma. Per esempio `gcc`, il compilatore C che si usa per tutto il corso:



Le **opzioni** modificano il comportamento del comando. Se non ricordi come si usa: `gcc --help` (sintesi rapida) oppure `man gcc` (manuale UNIX).

Gli argomenti li prepara la shell: **spezza la riga in parole usando gli spazi** e avvia il programma passandogli gli argomenti. Se un nome contiene spazi, le **virgolette** lo tengono insieme come un solo argomento:

```
$ touch "appunti lezione.txt"     # un file, non due
$ cat "appunti lezione.txt"
```

> Vedremo come un programma C legge questi argomenti con `argc` e `argv`, i parametri di `main`.

> **Esercizio — orientarsi**
> Stampare la directory corrente e il suo contenuto, creare una directory `lezione01`, entrarci e verificare di essere nel posto giusto. Comandi: `pwd`, `ls`, `mkdir`, `cd`.

1.2 Lavorare con i file

Per creare directory e file vuoti si usano `mkdir` e `touch`:

```
$ mkdir esercizi     # argomento = nome
$ touch appunti.txt  # file vuoto
```

`touch` in realtà nasce per **cambiare i timestamp** (gli orari registrati) di un file. Se il file non esiste lo crea: la creazione è una conseguenza utile, non lo scopo.

Oltre al contenuto, per ogni file UNIX tiene tre marcature temporali (**timestamp**). `stat file` le mostra, insieme agli altri metadati.

Metadato	Significato
`atime`	*access time*: ultimo accesso al **contenuto**
`mtime`	*modification time*: ultima modifica del **contenuto**
`ctime`	*change time*: ultima modifica dei **metadati**

> `ctime` vuol dire **change** time, **non** «creation time». Alcuni filesystem moderni salvano anche il tempo di creazione (*birth time*), ma non fa parte dei tre timestamp UNIX classici.

A cosa servono: backup incrementali, sincronizzazione, diagnostica, cercare i file modificati di recente, e ricompilare solo ciò che è cambiato. `make` (lo vedremo più avanti) confronta i timestamp per decidere se ricompilare.

Chi aggiorna cosa (✓ = portato all'ora corrente):

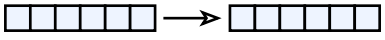
Operazione	`atime`	`mtime`	`ctime`
`touch file`	✓	✓	✓ (cambiare i tempi è già una modifica dei metadati)
`touch -a file`	✓	—	✓
`touch -m file`	—	✓	✓
`cp` : destinazione	✓ solo se creato	✓	✓
`cp -p` : destinazione	copiato dal sorgente	copiato dal sorgente	✓
`cp` : sorgente	✓ può, per la lettura	—	—

Per copiare, spostare e cancellare:

`cp sorgente destinazione` (*copy*): l'originale resta al suo posto.

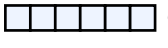
```
$ cp hello.c copia.c
$ cp /tmp/ciao.txt ~/hello.txt   # copia e rinomina
```

copia — $O(N)$



legge e riscrive ogni byte

sposta (stesso filesystem) — $O(1)$



cambia solo il **percorso** nei metadati

`mv` (*move*): un comando, **due usi**.

```
$ mv copia.c esempio.c      # rinomina
$ mv esempio.c esercizi/    # sposta
```

Operazione	Costo
copia	$O(N)$
sposta, stesso filesystem	$O(1)$
sposta, altro filesystem	$O(N)$: copia + cancella

Comando	Descrizione
`rm`	cancella un file (*remove*)
`rmdir`	cancella una directory **vuota**
`rm -r`	cancella una directory con tutto il contenuto. `-r` = **ricorsivamente**: attraversa tutto ciò che c'è dentro

> `rm` **non** sposta nel cestino: da terminale la cancellazione è in genere **definitiva**.

Per guardare dentro un file non serve sempre un editor:

Comando	Descrizione
`cat`	scrive tutto il contenuto del file sul terminale
`less`	per file lunghi: si scorre avanti e indietro e si cerca, senza riversare tutto sul terminale. `q` **per uscire**
`head` / `tail`	prime / ultime 10 righe. `head -n 5 dati.txt` = prime 5

Per lavorare su molti file insieme ci sono le **wildcard**, che espande la shell:

- `*` = **zero o più** caratteri qualsiasi
- `?` = **esattamente un** carattere

```
$ ls *.c
hello.c  main.c  prova.c
$ ls prova?.c
prova1.c  prova2.c  provaA.c
```

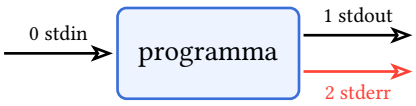


l'espansione avviene **prima** che `ls` parta: `ls` non vede mai l'asterisco

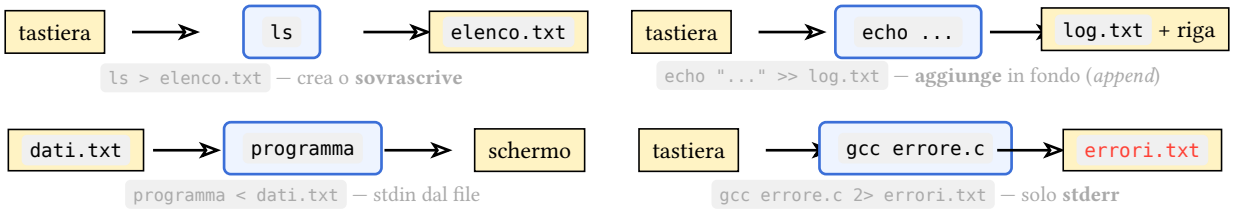
1.3 Flussi: redirezione e pipe

Ogni programma ha tre **flussi standard**:

n.	Flusso	Di solito
0	`stdin` — standard input	dalla tastiera
1	`stdout` — standard output	sul terminale
2	`stderr` — standard error	sul terminale



La shell può collegare ogni flusso a qualcosa di diverso dal terminale. Le **redirezioni** lo collegano a un file:




```
$ ls > elenco.txt              # sul terminale non compare niente
$ cat elenco.txt
appunti.txt
```

```
elenco.txt
esercizi
hello.c
$ echo "prima riga" > log.txt
$ echo "seconda riga" >> log.txt
$ cat log.txt
prima riga
seconda riga
```

Con `<` il programma continua a leggere da stdin: è **la shell** che ha collegato stdin al file, il programma non se ne accorge.
`2>` redirige il file descriptor 2. Si separa stderr da stdout perché output normale ed errori hanno significati diversi e spesso si trattano in modo diverso.

La **pipe** `|` invece collega lo **stdout del primo** comando allo **stdin del secondo**.



1. la shell espande `*.c`
2. `ls` produce l'elenco
3. la pipe lo passa a `wc`
4. `wc -l` conta le righe

Con un file temporaneo

```
$ ls > tmp.txt
$ wc -l < tmp.txt
```

Con la pipe: niente file in mezzo

```
$ ls | wc -l
$ ls | less      # scorro l'elenco
```

Idea UNIX: piccoli programmi che si combinano per costruire operazioni più complesse.

Esercizio — file e flussi
Partendo dalla home: creare la directory `lezione02` ed entrarci; creare tre file vuoti `a.c`, `b.c`, `note.txt`; mostrare solo i file `.c` e salvare quell'elenco in `sorgenti.txt`; aggiungere il contenuto di `sorgenti.txt` a `note.txt`; contare le righe di `note.txt`.

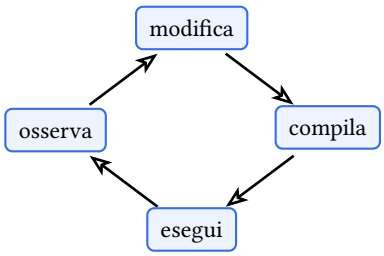
2 Il linguaggio C

Cosa devi saper fare

- compilare con `gcc`, separare le fasi (`-E`, `-c`, linking) e distinguere errori di compilazione, di linking e di esecuzione;
- dire tipo e valore di un'espressione: divisione intera, conversioni, casting, cortocircuito, `++` prefisso e postfisso;
- scrivere cicli, `if` e `switch`, e leggere e stampare con `scanf` e `printf`;
- usare array, matrici, `struct`, `enum` e `typedef`;
- scrivere funzioni con prototipo, dividere un programma in `.h` e `.c`, e dire dove vive e chi vede una variabile (`auto`, `extern`, `static`).

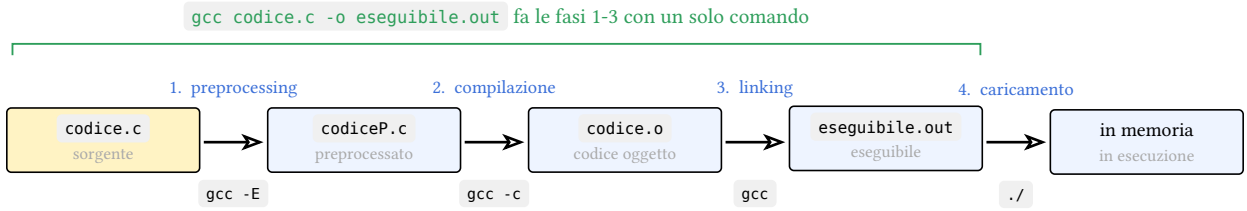
2.1 Dal sorgente all'eseguibile

Scrivere un programma C è un ciclo che si ripete:



> **modifica ≠ compilazione ≠ esecuzione.** Se cambi il sorgente e non ricompili, stai eseguendo il vecchio programma.

Il passo centrale è la compilazione. C è un linguaggio **compilato**: il sorgente `.c` è un semplice file di testo, e per diventare un programma che gira attraversa **quattro fasi**.



1. **Preprocessing**: vengono eseguite le **direttive per il pre-processore**, le righe che iniziano con `#`.
2. **Compilazione**: il sorgente viene tradotto in **codice oggetto** `.o`.
3. **Linking**: i file oggetto vengono collegati in un unico **eseguibile**.
4. **Caricamento**: quando lo lanci, l'eseguibile viene copiato in memoria e parte.

> **Da sapere — codice oggetto e linking** (aggiunto, non spiegato in aula)
> Il codice oggetto è il sorgente già tradotto in linguaggio macchina, ma non ancora eseguibile: gli mancano i pezzi che stanno altrove. Per esempio il codice di `printf` non è nel tuo file, sta nella libreria standard. Il **linker** mette insieme il tuo `.o` e questi pezzi, e ne esce un file che si può eseguire.

La prima fase lavora solo sul testo: le direttive sostituiscono un nome con il contenuto che indica. Due usi:

- `#include <stdio.h>` **include un file**: al suo posto viene incollato il contenuto di una libreria come `stdio.h`, o di un altro file come `my_lib.c`;
- `#define NOME testo` **definisce una macro**: da lì in poi ogni `NOME` viene sostituito con `testo`. Serve per costanti o comandi semplici.

Con `gcc -E` ci si ferma dopo il preprocessing e si vede cosa arriva al compilatore:

```c
#include <stdio.h>
#define CONST_MESSAGE "Hello world\n"

int main(void) {
    printf(CONST_MESSAGE);
    return 0;
}
```

```
$ gcc -E codice.c -o codiceP.c
$ wc -l codiceP.c
845 codiceP.c     # 7 righe diventate 845
$ tail -4 codiceP.c
int main(void) {
    printf("Hello world\n");
    return 0;
}
```

Le righe in più sono il contenuto di `stdio.h`, incollato dall'`include`. In fondo c'è il nostro `main`, con `CONST_MESSAGE` già sostituita dal suo testo.

Di solito le prime tre fasi si fanno con un solo comando, `gcc` (*GNU Compiler Collection*):

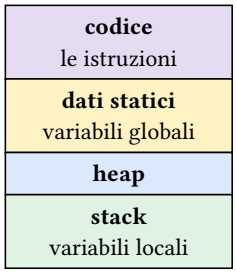
```
$ gcc hello.c              # se va tutto bene non scrive niente
$ ls
a.out  hello.c             # senza -o l'eseguibile si chiama a.out
$ gcc -Wall -Wextra -std=c11 hello.c -o hello   # warning, C11, nome hello
$ ./hello
Hello, world!
$ echo $?
0
```

Pezzo	Significato
`-o hello`	dà il nome all'eseguibile
`-Wall -Wextra`	attiva i warning: **non sono rumore**, spesso segnalano un errore vero
`-pedantic`	segnala tutto ciò che non rispetta alla lettera lo standard del C
`-g`	aggiunge all'eseguibile le informazioni per il *debugger* (il programma che serve a cercare gli errori)
`-std=c11`	standard del C da usare (esistono C89, C95, C99, C11, C17, C23)
`./hello`	«esegui `hello` **che sta nella directory corrente**». La shell cerca i comandi in un elenco di directory, e quella corrente non c'è per forza
`echo $?`	stampa il valore restituito dall'ultimo programma. **0 = terminato correttamente**

Le opzioni complete sono in `man gcc` e su gcc.gnu.org/onlinedocs/gcc. Volendo, le fasi si possono anche separare:

```
$ gcc -E codice.c -o codiceP.c     # 1. solo preprocessing
$ gcc -c codice.c -o codice.o      # 1+2: si ferma al codice oggetto
$ gcc -c codiceP.c -o codice.o     # oppure: compila il file già preprocessato
$ gcc codice.o -o eseguibile.out   # 3. linking
$ ./eseguibile.out                 # 4. caricamento ed esecuzione
Hello world
```

Nella quarta fase l'eseguibile viene caricato in memoria, divisa in quattro zone:



Variabili globali e locali sono spiegate nella prossima sezione.

Lo **stack** (pila) cresce e si svuota da solo: quando una funzione parte, le sue variabili locali vengono messe in cima; quando finisce, vengono tolte. L'**heap** è la memoria che il programma chiede esplicitamente mentre gira, e che resta finché non la restituisce (Sezione 3.3).

Se qualcosa va storto, l'errore può arrivare in due momenti diversi:

```c
#include <stdio.h>

int main(void) {
    puts("Hello, world!")   // manca ;
    return 0;
}
```

```
hello.c:4:26: error: expected ';' before
'return'
```

Errore di compilazione	Errore di esecuzione
il programma **non viene costruito**	l'eseguibile esiste ma **si comporta male** o termina in modo inatteso

File, riga, messaggio: usali. Correggi **dall'alto verso il basso**: un solo errore può generare molti messaggi dopo.

Se non funziona, controlla (sono gli errori banali che fanno perdere tempo):
1. sono nella directory giusta? (`pwd`)
2. il sorgente esiste, ed è quello che ho modificato? (`ls`)
3. ho ricompilato? ci sono errori o warning?
4. l'eseguibile è stato creato, e sto eseguendo quello giusto?

> **Esercizio — il ciclo completo**
> Creare `hello.c`, compilarlo con i warning attivi ed eseguirlo; poi cambiare il messaggio, ricompilare e rieseguire.

2.2 Com'è fatto un programma C

C segue il **paradigma imperativo**: il programma è una sequenza di istruzioni che, una dopo l'altra, cambiano i valori in memoria. Ed è **strutturato a blocchi**: il programma è una collezione di **funzioni**, e l'esecuzione parte sempre dalla funzione `main`. Un file C ha tre parti, in quest'ordine:

```c
#include <stdio.h>   // 1. direttive

int globalA;         // 2. globali

int main(void) {     // 3. funzioni
    puts("Hello!");
    return 0;
}
```

1. **direttive per il pre-processore**: `#include <stdio.h>` rende disponibili le funzioni di input/output (come `puts`);
2. **dichiarazioni globali**: variabili visibili in tutto il programma;
3. **funzioni**: `main` prende niente (`void`) e restituisce un intero (`int`). `return 0` la termina e restituisce 0 al sistema operativo.

Graffe, commenti (`/* … */` e `//`) e punto e virgola sono come in JavaScript.

Per stampare ci sono due funzioni, `puts` e `printf`:

Funzione	Cosa fa
`puts("Laboratorio 2");`	stampa una stringa **e va a capo**
`printf("Risultato: %d\n", 42);`	stringa **formattata**: `%d` viene sostituito dal valore. **Non va a capo da sola**: serve `\n`

> `printf` è una funzione a **numero variabile di parametri**: il primo è la stringa di formato, gli altri i valori da inserire al posto dei *placeholder* `%d`, `%s`, …

Mettendo insieme i pezzi:

```c
#include <stdio.h>

int main(void) {
    int x = 21;   // una variabile

    puts("Laboratorio 2");
    puts("Lezione 1");
    printf("2 * %d = %d\n", x, 2 * x);

    return 0;
}
```

```
$ gcc -Wall -Wextra -std=c11
lezione01.c -o lezione01
$ ./lezione01
Laboratorio 2
Lezione 1
2 * 21 = 42
```

> **Esercizio — tre modifiche**
> Partendo da `hello.c`: stampare il proprio nome; stampare due righe con `printf`; introdurre una variabile intera e stamparne il valore.

I nomi che scegli tu (di variabili, funzioni, tipi) sono **identificatori**: sequenze di lettere, cifre e `_` che iniziano con una lettera o con `_`. Le **parole chiave** del linguaggio (`int`, `return`, `while`, …) sono riservate e non si possono usare come nomi.

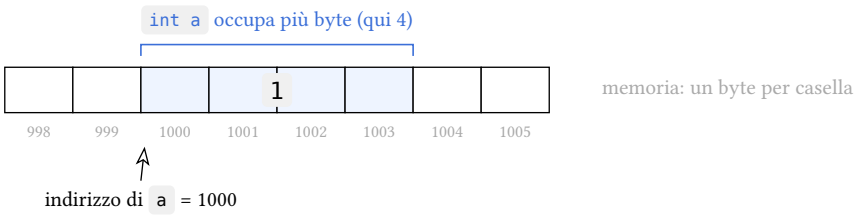
Nome	Valido	Perché
`somma`	sì	lettere
`_tmp`	sì	inizia con `_`
`x1`	sì	la cifra non è all'inizio
`1x`	no	inizia con una cifra
`mia-var`	no	`-` non è ammesso
`int`	no	parola chiave

Nel programma di sopra `x` è una **variabile**: una **locazione di memoria** che contiene un **valore modificabile** di un certo **tipo**. L'insieme dei valori di tutte le variabili in un certo momento è lo **stato del programma**.

```c
int a;        // dichiarazione
int a = 1;    // con valore iniziale
```

La **dichiarazione** dice al compilatore il tipo, e quindi **quanta memoria** riservare.

A ogni variabile è associato un **indirizzo**: un numero intero che indica dove si trova in memoria.



Una variabile però non è utilizzabile ovunque. Lo **scoping** è la regione del programma in cui una variabile è dichiarata, definita e utilizzabile: fuori da lì non è visibile. In C lo scoping è **statico**: viene deciso **a tempo di compilazione**, guardando dove sta la dichiarazione nel testo.

Globale: dichiarata fuori da ogni blocco e funzione. Sta nei **dati statici** ed è visibile nell'intero programma.

Locale: dichiarata dentro un blocco o una funzione. Sta sullo **stack** ed è visibile nel blocco e nei blocchi annidati dentro.

Una variabile locale in un blocco annidato può **coprire** una variabile con lo stesso nome, globale o di un blocco più esterno:

```c
int g = 1;              // globale

int main(void) {
    int x = 5;          // locale a main
    {
        int g = 2;  // copre la g globale
        printf("g = %d, x = %d\n", g, x);
    }
    printf("g = %d\n", g);
    return 0;
}
```

programma — vede `g = 1`

`main` — vede `g = 1` e `x`

blocco interno — vede `x` e la **sua** `g = 2`
la `g` globale qui è coperta

```
$ ./scope
g = 2, x = 5     # dentro il blocco
g = 1            # fuori
```

2.3 Tipi e costanti

C è **strongly typed**: ogni espressione e ogni variabile ha un tipo, noto al compilatore. I tipi di base sono quattro:

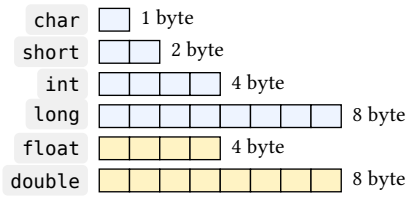
Tipo	Contiene
`char`	un intero grande **un byte**
`int`	numeri interi: `123`, `-45`, `0`; anche costanti carattere come `'a'` o `'1'`
`float`	numeri reali
`double`	numeri reali in **precisione doppia**

Si possono modificare con quattro **qualifier**:

- `short` e `long` qualificano gli interi: `short int a;`. Da soli stanno per `short int` e `long int`. `long` può qualificare anche `double`;
- `signed` e `unsigned` qualificano `char` e `int`: `unsigned` = solo valori ≥ 0.

Quanti byte occupa ogni tipo **dipende dalla macchina**: il C garantisce solo `short` ≤ `int` ≤ `long`. Per saperlo si usa l'operatore `sizeof`, che restituisce la dimensione in byte:

```c
int main(void) {
    int a = sizeof(int);
    printf("%d\n", a);    // stampa 4
    return 0;
}
```



Misurate così su questo computer (Linux a 64 bit):

`sizeof` si applica anche alle **espressioni**, e lì c'è una sorpresa (`%zu` è il placeholder di `printf` per il risultato di `sizeof`):

```c
short int a = 1;
printf("%zu\n", sizeof(a));      // 2
printf("%zu\n", sizeof(a + 1));  // 4
```

Nei calcoli i valori `char` e `short` vengono prima convertiti in `int`. Quindi `a + 1` è un `int`, anche se `a` è `short`: 4 byte, non 2.

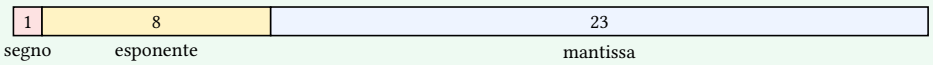
Il `char` è un intero, ma si usa per i **caratteri**. La tabella **ASCII** assegna un carattere a ogni valore che sta in un byte:

`'a'`	`'b'`	`'*'`	`'A'`	`'0'`
97	98	42	65	48

Quindi `'a'` e `97` sono lo stesso numero: cambia solo come lo stampi (`%c` o `%d`). Nota `'0'` = 48: il carattere zero non è il numero zero.

> **Da sapere — `'a'` non è un `char`** (aggiunto, non spiegato in aula)
> Una costante carattere come `'a'` in C ha tipo `int`: `sizeof('a')` dà 4, mentre `sizeof(char)` dà 1. Il valore è lo stesso (97), solo che occupa più spazio finché non lo metti in una variabile `char`.

I reali `float` e `double` sono rappresentati in **notazione scientifica**, secondo lo standard **IEEE 754**. Tre componenti: il **bit di segno**, l'**esponente** e la **mantissa**, normalizzata in modo che la parte intera sia 1. Per esempio $1.34423424 \cdot 10^{-5}$ rappresenta 0.0000134423424.

> **Da sapere — quanti bit per ogni parte** (aggiunto, non spiegato in aula)
> In un `float` (32 bit) le tre parti sono grandi così; in un `double` (64 bit) sono 1, 11 e 52 bit.
>
> 
>

Non tutti i numeri sono rappresentabili: i bit sono finiti, quindi molti numeri vengono approssimati al più vicino disponibile. Anche 0.1:

```
$ ./reali                          # printf("%.20f\n", 0.1);
0.10000000000000000555
```

Le **costanti** sono valori non modificabili scritti direttamente nel codice: interi, in virgola mobile, caratteri e stringhe.

Costante	Esempi
intera o in virgola mobile, anche con segno	`1`, `-32`, `3.15`
intera in ottale (inizia con `0`) o esadecimale (inizia con `0x`)	`014`, `0x3A`
carattere, tra apici singoli	`'a'`, `'\n'`
stringa, tra apici doppi	`"ciao!\n"`

Tutte le costanti in virgola mobile sono `double`.

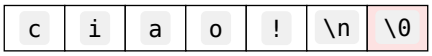
> **Da sapere — ottale ed esadecimale** (aggiunto, non spiegato in aula)
> Sono lo stesso numero scritto in base 8 o in base 16 invece che in base 10. In esadecimale le cifre dopo il 9 sono `A` =10, `B` =11, ..., `F` =15.
> - `014` $= 1 \cdot 8 + 4 = 12$
> - `0x3A` $= 3 \cdot 16 + 10 = 58$
>
> Attenzione: lo `0` iniziale non è decorativo. `010` vale 8, non 10.

Il carattere `\` è un carattere speciale di **escape**: insieme al carattere dopo indica un carattere che non si può scrivere direttamente, perché non stampabile o già usato dalla sintassi.

`'\n'`	`'\t'`	`'\0'`	`'\\'`
a capo	tabulazione	carattere nullo (codice 0)	la barra `\` stessa

Le costanti stringa sono memorizzate come **array di caratteri**, cioè caratteri uno dopo l'altro in memoria:

In fondo a ogni stringa il compilatore aggiunge `'\0'`, che segna dove finisce (Sezione 3.5). Per questo `sizeof("ciao!\n")` vale 7: sei caratteri più il terminatore.



Ogni tipo ha il suo placeholder in `printf`, e **il placeholder deve essere compatibile con il tipo del valore**. Ognuno è fatto così:



`printf("%08.2f", 3.14);` → `00003.14` — forma generale: `%[flags][width][.precision][length]specifier`

Interi		Altri tipi	
`%d`, `%i`	int con segno	`%f`	double
`%u`	unsigned int	`%e`, `%E`	notazione scientifica
`%x`, `%X`	esadecimale	`%g`, `%G`	formato compatto
`%o`	ottale	`%c`	carattere
`%ld`	long	`%s`	stringa
`%lld`	long long (ancora più lungo)	`%p`	indirizzo di memoria
`%zu`	`size_t`: il risultato di `sizeof`	`%%`	il carattere %
`%hd`	short	`%Lf`	long double

`printf` scrive su stdout; per **leggere da stdin** c'è `scanf`, sempre nella libreria `stdio`. La stringa di formato è la stessa, ma dopo si passano gli **indirizzi** delle variabili in cui salvare i valori: `&a` è l'indirizzo di `a`.

```c
int a, b;
scanf("%d %d", &a, &b);   // due interi
printf("a = %d\n", a);
```

I placeholder sono gli stessi di `printf`, **tranne quello per `double`**: in `scanf` è `%lf`.

Ogni carattere in più nella stringa di formato `scanf` **lo aspetta in input**: con `scanf("ciao%d", &n)` bisogna scrivere proprio `ciao` prima del numero. Per questo il prof chiede una `scanf` per variabile, con solo il placeholder.

2.4 Operatori e controllo del flusso

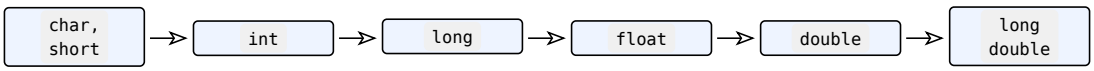
Operatori (`+ - * / %`, `== != < <= > >=`, `! && ||`, `= += -= …`, `++ --`), `if`/`else`, `while`, `do-while`, `for`, `break`, `continue` e `switch` si scrivono e funzionano come in JavaScript, cortocircuito di `&&` e `||` compreso. Le differenze sono nei **tipi**.

La prima: **tra due interi la divisione è intera**, la parte decimale si butta. In JavaScript `5 / 3` darebbe 1.666…

```c
5 / 3        // 1          divisione intera
5.0 / 3.0    // 1.666667   divisione tra reali
5 % 3        // 2          resto
```

Il risultato deve stare nel tipo dell'espressione: se è troppo grande (o troppo piccolo) per i byte a disposizione si ha **overflow** (o **underflow**), e il valore che esce è sbagliato.

Se in un'espressione ci sono tipi diversi, il C li converte da solo (**conversione implicita**): il tipo più «corto» viene trasformato in quello più «lungo», lungo questa scala (è la stessa promozione vista con `sizeof(a + 1)`).



Nell'assegnamento invece **vince il tipo della variabile a sinistra**: se è più corto del valore, si può perdere informazione. Per scegliere noi il tipo c'è la **conversione esplicita**, con l'operatore di **casting**: `(tipo) espressione`. Con `int a = 5;`:

Assegnamento	`b` vale	Perché
`float b = a;`	`5.000000`	l'intero viene promosso a `float`
`float b = a / 2;`	`2.000000`	due interi: divisione **intera**, e solo dopo la conversione
`float b = a / 2.0;`	`2.500000`	`2.0` è reale: `a` viene convertito, divisione tra reali
`float b = (float) a / 2;`	`2.500000`	il casting cambia il tipo di `a` **prima** della divisione
`float b = (float) (a / 2);`	`2.000000`	qui il casting si applica al **risultato**: la divisione è già stata fatta fra interi

Lo stesso casting serve nei due versi. Per avere il quoziente reale di due interi si converte un operando; per usare `%`, che vuole interi, su un valore letto come `float` lo si converte in `int`:

```c
int a, b;
scanf("%d %d", &a, &b);        // 7 2
printf("%d\n", a / b);           // 3
printf("%.2f\n", (float) a / b); // 3.50
```

```c
float valore;
scanf("%f", &valore);   // 7.0
int n = (int) valore;
if (n % 2 == 0) puts("Pari");
else puts("Dispari");
```

La seconda: in C **non c'è un tipo booleano di base**. **0 è falso, qualsiasi valore diverso da 0 è vero**, e un confronto o un operatore logico restituisce un `int`: **0 se falso, 1 se vero**. Le condizioni di `if` e dei cicli sono quindi numeri: `while (1)` è un ciclo infinito, `if (n)` vuol dire `if (n != 0)`.

```
a = 7          2 < a < 5
prima 2 < a        1 < 5
risultato      1   (vero!)
```

Trappola: `2 < a < 5` **non** controlla che `a` stia fra 2 e 5. È `(2 < a) < 5`, e `2 < a` vale 0 o 1, sempre minore di 5. Si scrive `2 < a && a < 5`. Con `-Wall` gcc avvisa: `comparisons like 'X<=Y<=Z' do not have their mathematical meaning`.

L'ultima riguarda lo `switch`: l'espressione deve essere **intera** (niente stringhe, a differenza di JS), e le etichette dei `case` costanti e diverse fra loro. Come in JavaScript, **senza `break` si prosegue nei `case` successivi**: con `c = 2`

```c
case 1: puts("uno");
case 2: puts("due");   ← entra qui; niente break: prosegue ↓
case 3: puts("tre"); break;   → esce
default: puts("altro");
```

stampa `due` e `tre`. Con `-Wextra` gcc avvisa: `this statement may fall through`.

> **Esercizio — sequenza terminata da 0**
> Leggere una sequenza di interi che termina con 0 (lo 0 non fa parte della sequenza). Stampare quanti numeri sono stati inseriti, quanti sono positivi, quanti negativi e la somma dei positivi.

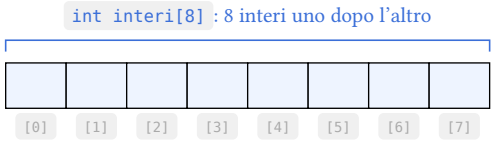
> **Esercizio — indovina il numero**
> Fissare nel programma un intero, per esempio 37. Chiedere ripetutamente un tentativo all'utente e stampare se è troppo grande, troppo piccolo o corretto; terminare quando il numero è indovinato.

2.5 Array e tipi custom

Un **array** è una collezione di valori **tutti dello stesso tipo**. Per definirlo bisogna conoscerne la **dimensione**; alla dichiarazione viene riservato uno spazio **contiguo** in memoria, di solito sullo stack. Si accede a un elemento con l'operatore `[]`: `a[i]`, con `i` che parte da 0.

```c
int interi[8];

#define N 100
double reali[N];   // dimensione con una macro
```



Un array si scorre con un `for` sull'indice. Per esempio il **prodotto scalare** di due vettori, cioè la somma dei prodotti degli elementi con lo stesso indice:

```c
#define N 10
int a[N], b[N], prodotto = 0, i;

for (i = 0; i < N; i++)
    scanf("%d", &a[i]);     // leggo a
for (i = 0; i < N; i++)
    scanf("%d", &b[i]);     // leggo b
for (i = 0; i < N; i++)
    prodotto += a[i] * b[i];
```

`&a[i]` è l'indirizzo della cella `i`: `scanf` scrive lì.

`prodotto` alla fine vale `a[0]*b[0] + … + a[N-1]*b[N-1]`: l'ultimo indice è `N-1`, non `N`.

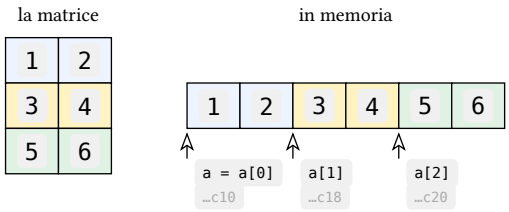
> **Esercizio — invertire un array**
> Invertire un array `A` di 10 interi **senza array d'appoggio**: scambiare la prima cella con l'ultima, la seconda con la penultima, e così via. Poi stampare ogni elemento dell'array modificato.

Un array può avere più dimensioni. `int a[3][3];` riserva 9 interi contigui, e si usa per rappresentare una **matrice**. In memoria non c'è una griglia: gli array a più dimensioni sono salvati **per righe**, una riga dopo l'altra.

```c
int a[3][2] = {{1,2},{3,4},{5,6}};

printf("%p\n", a);
printf("%p, %p, %p\n", a[0], a[1], a[2]);
```

```
0x7ffd2ba5ac10
0x7ffd2ba5ac10, 0x7ffd2ba5ac18, 0x7ffd2ba5ac20
```



`a`, `a[0]`, `a[1]` e `a[2]` sono gli **indirizzi delle righe**: `a` coincide con `a[0]`, e ogni riga inizia 8 byte dopo la precedente, cioè 2 `int` da 4 byte. Per questo la **seconda dimensione** è importante: per trovare `a[i][j]` bisogna saltare `i` righe intere, e serve sapere quanto è lunga una riga. Le dimensioni possono essere anche più di due: `int a[3][4][2];`.

Una matrice si scorre con due `for` annidati, uno sulle righe e uno sulle colonne. Il prodotto scalare di prima, fra due matrici 10×10:

```c
int a[N][N], b[N][N], prodotto = 0, i, j;
// ... lettura: scanf("%d", &a[i][j]) dentro gli stessi due for
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        prodotto += a[i][j] * b[i][j];
```

Gli array tengono valori dello stesso tipo. Per mettere insieme valori di **tipo diverso** c'è la `struct`. La parola chiave `struct` **definisce un nuovo tipo, non dichiara una variabile**: la variabile si dichiara dopo. I pezzi si chiamano **membri** e si leggono con `variabile.membro`.

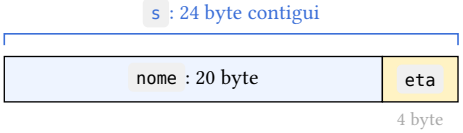
```c
#define LEN 20

struct Studente {     // il nuovo tipo
    char nome[LEN];
    int eta;
};

int main(void) {
    struct Studente s = {"Antonio", 15};

    printf("Nome: %s, Età: %d\n",
           s.nome, s.eta);
    printf("Dimensione: %zu\n",
           sizeof(s));
    return 0;
}
```

```
$ ./main
Nome: Antonio, Età: 15
Dimensione: 24
```



L'ordine dei membri nella definizione è l'ordine in memoria, e lo spazio di una variabile è contiguo. I valori fra graffe vanno **nello stesso ordine dei membri**.

> Con l'ordine sbagliato, `{15, "Antonio"}`, il programma stampa `Nome: T, Età: 0`. Il compilatore avvisa: `incompatible pointer to integer conversion` (con un gcc recente è un errore, e l'eseguibile non viene creato).

> Il compilatore può inserire **padding** fra i campi, cioè byte vuoti, per rispettare vincoli di allineamento: `sizeof` di una `struct` può essere più della somma dei membri.

Il secondo tipo custom è l'`enum`, un tipo **enumerativo**: un elenco di nomi, ognuno dei quali è una **costante intera**. Serve a rendere il codice più leggibile, per esempio nei `case` di uno `switch`. Di default i valori partono da 0 e crescono di 1; si possono cambiare, e da lì in poi si continua a contare.

```c
enum giorni {lunedi, martedi, mercoledi, giovedi, venerdi, sabato, domenica};
```

lunedi	martedi	mercoledi	giovedi	venerdi	sabato	domenica
0	1	2	3	4	5	6

```c
enum giorni {lunedi = 1, martedi, mercoledi, giovedi = 7, venerdi, sabato, domenica};
```

lunedi	martedi	mercoledi	giovedi	venerdi	sabato	domenica
1	2	3	7	8	9	10

Infine `typedef` **dà un nuovo nome a un tipo esistente**: utile per nomi di tipo lunghi, o per averne uno più facile da ricordare. Senza `typedef` le parole `struct` ed `enum` vanno ripetute ogni volta che si dichiara una variabile.

```c
struct Studente { ... };
enum giorni { ... };

struct Studente s;   // senza typedef
enum giorni giorno;
```

```c
typedef struct { ... } Studente;
typedef enum { ... } Giorni;

Studente s;          // con typedef
Giorni giorno;
```

L'esempio che li usa tutti: l'anagrafe di `N` dipendenti. Di ognuno si leggono codice, età, stipendio e ruolo (0 dirigente, 1 quadro, 2 operaio, 3 impiegato); poi si stampano codice, età e ruolo di chi guadagna **meno dello stipendio medio**. I dipendenti stanno in un **array di struct**, il ruolo è un `enum`.

```c
#define N 10

typedef enum {
    dirigente,
    quadro,
    operaio,
    impiegato
} ruolo_t;

typedef struct {
    int id;
    int eta;
    float salario;
    ruolo_t ruolo;
} dipendente_t;
```

```c
dipendente_t dipendenti[N];
int i, ruolo_dip;
float media = 0;

for (i = 0; i < N; i++) {
    scanf("%d", &dipendenti[i].id);
    scanf("%d", &dipendenti[i].eta);
    scanf("%f", &dipendenti[i].salario);
    scanf("%d", &ruolo_dip);
    dipendenti[i].ruolo = ruolo_dip;
    media += dipendenti[i].salario;
}
media /= N;
```

`dipendenti[i].salario` si legge da sinistra: la cella `i` dell'array, e di quella il membro `salario`. La stampa usa uno `switch` sul ruolo, perché l'`enum` è un intero e `printf` stamperebbe solo il numero:

```c
for (i = 0; i < N; i++)
    if (dipendenti[i].salario < media) {
        printf("%d %d ", dipendenti[i].id, dipendenti[i].eta);
        switch (dipendenti[i].ruolo) {
            case dirigente: printf("dirigente\n"); break;
            case quadro:    printf("quadro\n");    break;
            case operaio:   printf("operaio\n");   break;
            default:        printf("impiegato\n");
        }
    }
```

```
3 22 operaio
4 32 impiegato
```

2.6 Funzioni e programmi su più file

Le funzioni si usano come in JavaScript, con due differenze: **tipo di ritorno e tipi dei parametri vanno scritti**, e non si possono definire funzioni dentro altre funzioni. La prima eseguita è sempre `main`.

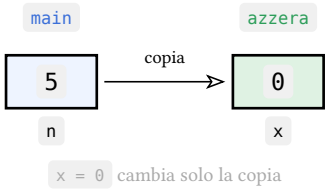
```c
int max(int a, int b) {
    if (a > b) return a;
    return b;
}
void saluta(void) {   // niente valore
    puts("ciao");
}
```

- i parametri attuali vengono convertiti al tipo dei formali (casting implicito), così come il valore di `return` al tipo di ritorno;
- se una funzione non `void` arriva in fondo **senza** `return`, chi usa il risultato legge un valore a caso.

> **Definizione — passaggio per valore.** In C i parametri sono passati **per valore**: la funzione riceve una **copia**. Cambiare il parametro cambia la copia, non la variabile del chiamante. Vale anche per le `struct`, che in JavaScript invece sarebbero oggetti condivisi.

```c
void azzera(int x) {
    x = 0;              // cambia la copia
}

int main(void) {
    int n = 5;
    azzera(n);
    printf("%d\n", n);  // 5
}
```



Per cambiare davvero una variabile del chiamante servono i puntatori (Sezione 3.1).

Il compilatore legge il file dall'alto in basso, e quando incontra una chiamata deve già sapere com'è fatta la funzione. Se la definizione sta più in basso, o in un altro file, si mette prima un **prototipo** (o **dichiarazione**): tipo di ritorno, nome e tipi dei parametri, senza corpo.

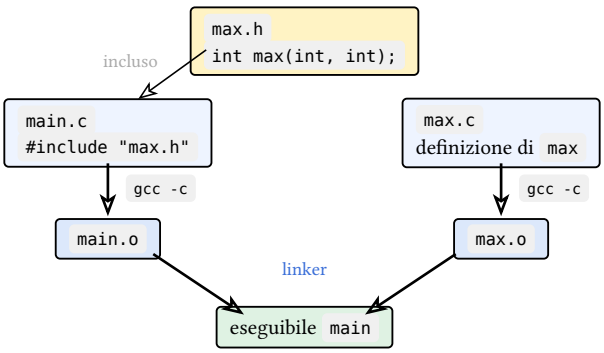
```c
int max(int, int);      // prototipo

int main(void) {
    printf("%d\n", max(3, 8));
}

int max(int a, int b) { // definizione
    if (a > b) return a;
    return b;
}
```

Manca	Errore di chi
la **dichiarazione**	del **compilatore**: `implicit declaration of function 'max'`
la **definizione**	del **linker**: `undefined reference to 'max'`

Questa differenza è la chiave dei programmi su **più file**. Per modularità e riuso il programma si divide in più file `.c`, che si compilano **separatamente** (ognuno diventa un `.o`) e poi il linker unisce. Le dichiarazioni di funzioni e costanti si raccolgono in un **file header** `.h`, incluso da chi le usa; le definizioni restano nei `.c`. `#include "max.h"`, con le virgolette, cerca il file nella directory corrente; con `< >` lo cerca fra le librerie di sistema.




```
$ gcc -Wall main.c max.c -o main    # compila i due file e li collega
```

Con le funzioni e i file cambia anche la vita delle variabili. Ogni variabile ha una **classe di memorizzazione**, che decide **dove** sta e **chi** la vede (lo scope visto in Sezione 2.1 e nella sezione sui programmi C):

Variabile	Dove sta	Chi la vede	Quanto vive
locale (classe `auto`, implicita)	stack	il suo blocco, fino alla `}`	allocata all'ingresso del blocco, tolta all'uscita
globale (`extern` implicito)	dati statici	il suo file; gli altri file se la dichiarano `extern`	tutto il programma
globale `static`	dati statici	**solo** il suo file	tutto il programma
locale `static`	dati statici	il suo blocco	tutto il programma: **mantiene il valore fra una chiamata e l'altra**

Le funzioni sono sempre globali: non si può definire una funzione dentro un'altra. `static` ha quindi due usi diversi: su una globale **impedisce l'uso da altri file**; su una locale la rende **persistente**.

L'esempio del prof li mostra tutti. `max.c` conta quante volte vince il primo numero (`countA`) e quante il secondo (`countB`); `main.c` legge coppie fino a due numeri uguali e alla fine vuole stampare `countA`:

```c
// max.c
int countA = 0;
int countB = 0;

int max(int a, int b) {
    if (a > b) {
        countA++;
        return a;
    }
    countB++;
    return b;
}
```

```c
// main.c
#include <stdio.h>
#include "max.h"

extern int countA;

int main(void) {
    int a = 0, b = 0;
    do {
        int a = 0;
        scanf("%d %d", &a, &b);
        printf("Max(%d,%d)=%d\n",
               a, b, max(a, b));
    } while (a != b);
    printf("a>b %d volte\n", countA);
}
```

Versione	Cosa succede
senza `extern int countA;`	errore del **compilatore**: `'countA' undeclared`. `main.c` non sa che `countA` esiste
con `extern int countA;`	compila: `extern` dice «esiste, ma è definita in un altro file»
con `extern`, ma in `max.c` `static int countA = 0;`	errore del **linker**: `undefined reference to 'countA'`. La `static` non si vede fuori da `max.c`

Quando ci fermiamo? Dentro il `do` c'è un'altra `int a = 0`, che **copre** la `a` esterna. `scanf` scrive nella `a` interna, ma la condizione `a != b` sta fuori dal blocco e usa la `a` **esterna**, che resta 0. Quindi `3 3` **non** ferma il ciclo: si esce solo con `b = 0`.

```
3 3
Max(3,3)=3          # continua!
0 0
```

```
Max(0,0)=0
a>b 2 volte
```

Ultima versione: i contatori diventano locali `static` dentro `max`, che li stampa a ogni chiamata. L'inizializzazione `= 0` avviene una volta sola, e le variabili non spariscono a fine chiamata: contano su tutte le chiamate.

```c
int max(int a, int b) {
    static int countA = 0;
    static int countB = 0;
    if (a > b) {
        countA++;
        printf("countA=%d, countB=%d\n",
               countA, countB);
        return a;
    }
    countB++;
    printf("countA=%d, countB=%d\n",
           countA, countB);
    return b;
}
```

```
3 5
countA=0, countB=1
Max(3,5)=5
6 2
countA=1, countB=1
Max(6,2)=6
3 8
countA=1, countB=2
Max(3,8)=8
```

Nota l'ordine delle righe: `countA=…` esce **prima** di `Max(…)`, perché i parametri di `printf` (fra cui `max(a, b)`) vengono calcolati prima che `printf` stampi.

3 Puntatori e memoria

Cosa devi saper fare

- leggere e scrivere `&x`, `*p`, `p + i`, `a[i]`, e prevedere quando il compilatore dà errore (puntatore ↔ intero, tipi diversi, `void*`);
- scrivere una funzione che modifica le variabili del chiamante (passaggio per indirizzo);
- allocare con `malloc` e `calloc`, controllare `NULL`, liberare con `free`, e riconoscere memory leak e uso dopo `free`;
- costruire una matrice dinamica per righe e liberarla;
- lavorare con le stringhe: terminatore, `char s[]` contro `char *s`, `string.h`, `strtok`, `ctype.h`, lettura e conversione;
- usare un puntatore a funzione e `qsort`;
- cercare un errore con `stderr`, `assert`, `gdb` e `valgrind`.

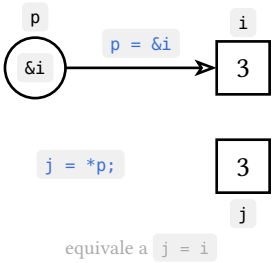
3.1 Puntatori

Tutto quello che sta in memoria, dati e istruzioni, ha un **valore** e un **indirizzo**: lo abbiamo già usato con `&a` in `scanf` e con `%p` in `printf`. Un indirizzo è un numero intero che dipende dalla macchina.

Definizione — puntatore. Un **puntatore** è una variabile che contiene un indirizzo di memoria. `int *p;` dichiara un puntatore a `int`: il **tipo** dice che cosa ci si aspetta di trovare a quell'indirizzo. Si inizializza con l'indirizzo di un'altra variabile, oppure con `NULL`, che vuol dire «nessun indirizzo».

Il puntatore è a sua volta una variabile, quindi ha anche lui un indirizzo. Due operatori lo collegano alla variabile puntata:

Operatore	Cosa fa
`&x`	**indirizzo** di `x`
`*p`	**dereferenziazione**: il valore che sta all'indirizzo contenuto in `p`. Si può leggere e scrivere



Il cerchio è il puntatore `p`, che contiene l'indirizzo di `i` (la freccia); `j = *p` va a leggere **ciò che sta in quell'indirizzo**, cioè il 3 di `i`. Lo stesso vale in scrittura, e più puntatori possono puntare alla stessa variabile:

```c
double a = 0;
double *aPtr = &a, *aPtr1 = &a;

*aPtr = 10;
printf("%f\n", a);   // 10.000000
*aPtr1 = 20;
printf("%f\n", a);   // 20.000000
```

Nella dichiarazione l'asterisco va ripetuto per ogni puntatore: `double *aPtr, *aPtr1`. Nell'espressione `*aPtr = 10` invece l'asterisco è la dereferenziazione: scrive 10 **dentro** `a`.

Un puntatore contiene un numero intero, ma **il C non lo tratta come un intero**: si usa nelle espressioni fra puntatori, non mescolato con altri tipi interi.

```c
int b = 1;
int c = 22;
int *a = &c;
*a = b + *a;    // ok: cambia c, che diventa 23
a = b + *a;     // NO: a è un puntatore, non un intero
*a = b + a;     // NO: b + a è un puntatore, e *a è un int
int *a = c;     // NO: a un puntatore non si assegna un intero
```

Le tre righe sbagliate danno errore: `makes pointer from integer without a cast` o `makes integer from pointer without a cast`.

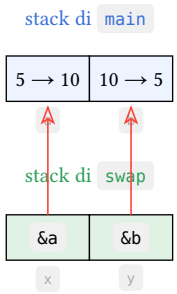
> **Il tipo del puntatore conta**: dice quanti byte leggere e come interpretarli. Puntare un `double` con un `int *` (o il contrario) dà il warning `incompatible pointer types` (con un gcc recente è un errore), e il valore letto è senza senso:
>
> ```c
> int a = 15;
> double *aPtr = &a;
> printf("%f\n", *aPtr);  // 0.000000
> ```
>
> ```c
> double a = 15.6;
> int *aPtr = &a;
> printf("%d\n", *aPtr);  // 858993459
> ```

> Dereferenziare un puntatore che vale `NULL`, o che punta a memoria non allocata, dà un **errore a tempo di esecuzione**: compila, ma quando gira termina con `Segmentation fault`.

Ecco a cosa servono. I parametri di una funzione vengono allocati sullo stack della funzione chiamata, e sono copie (Sezione 2.6). Ma se le si passa l'**indirizzo** di una variabile, la funzione può modificare quella variabile, anche se sta nell'ambiente del chiamante. È il **passaggio per indirizzo**, ed è il motivo per cui `scanf` vuole `&a`.

```c
void swap(int *x, int *y) {
    int var = *x;
    *x = *y;
    *y = var;
}

int main(void) {
    int a = 5, b = 10;
    printf("a=%d,b=%d\n", a, b);  // a=5,b=10
    swap(&a, &b);
    printf("a=%d,b=%d\n", a, b);  // a=10,b=5
}
```

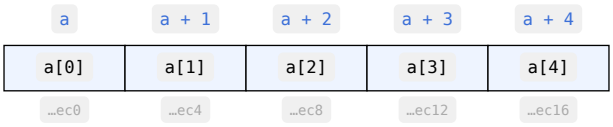


`x` e `y` sono copie degli **indirizzi** (le frecce rosse), ma `*x` e `*y` sono proprio `a` e `b`. Con `void swap(int x, int y)` lo scambio avverrebbe sulle copie, e `a`, `b` resterebbero 5 e 10.

3.2 Array e puntatori

Un array è un insieme **contiguo** di elementi dello stesso tipo. `int a[10];` fa tre cose: crea una variabile `a`, alloca sullo stack 10 interi contigui, e mette in `a` l'indirizzo dell'inizio di quella zona. Per questo il prof dice che `a` è «in realtà un puntatore»: vale `a == &a[0]`.

Dato l'indirizzo del primo elemento, gli altri si trovano con l'**aritmetica dei puntatori**. Sommare `i` a un puntatore vuol dire **avanzare di `i` elementi**, non di `i` byte: il compilatore moltiplica da solo per la dimensione del tipo.



`int` : 4 byte ciascuno

In C si scrive	È uguale a	Indirizzo in byte
`a + i`	`&a[i]`	indirizzo di `a` + `i · sizeof(int)`
`*(a + i)`	`a[i]`	
`&a[i][j]`, matrice con `n_col` colonne		indirizzo di `a` + `(i · n_col + j) · sizeof(int)`

> Attenzione a non moltiplicare due volte. In C si scrive `a + i`, **non** `a + i * sizeof(int)`: quest'ultimo avanzerebbe di `4i` elementi. La moltiplicazione per `sizeof` serve solo a calcolare l'indirizzo in byte, come nell'ultima colonna. Lo conferma la stampa: `a` e `a + 1` distano 4 byte.

Quando si passa un array a una funzione **viene passato il puntatore al primo elemento**, non una copia. Quindi:

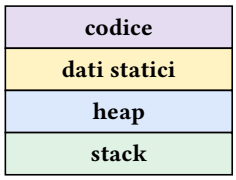
- un parametro di tipo puntatore si usa come un array: `int *v` e `int v[]` come parametro sono la stessa cosa, e `v[i]` funziona;
- la funzione **può modificare** l'array del chiamante;
- la funzione non sa quanto è lungo: la dimensione va passata a parte.

Per un array a più dimensioni bisogna scrivere **tutte le dimensioni tranne la prima**, perché servono per saltare le righe (come visto per le matrici): `int f(int a[][5][3]) {...}`.

> `a` **non** è però una variabile puntatore come le altre. `sizeof(a)` dà 40 (tutto l'array), mentre `sizeof` di un `int *` dà 8. E `a` non si può riassegnare: `a = b;` fra due array dà `assignment to expression with array type`.

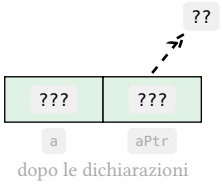
3.3 Allocazione statica e dinamica

Finora la memoria la decideva il compilatore. Vediamo meglio le zone in cui è divisa (Sezione 2.1):



- **dati statici**: quello che si sa già a tempo di compilazione, cioè le variabili globali e le locali `static`;
- **stack**: un **record di attivazione** per ogni blocco di codice, comprese le chiamate di funzione, con lo spazio per le variabili locali;
- **heap**: dati **dinamici**, allocati a tempo di esecuzione e raggiungibili tramite puntatori. È **frammentato**: i pezzi non sono per forza vicini.

Partiamo da due dichiarazioni:



dopo le dichiarazioni

- `int a;` alloca `sizeof(int)` byte. Il valore è **non definito**.
- `int *aPtr;` alloca `sizeof(int*)` byte per il puntatore. Sono non definiti sia il suo valore (a cosa punta) sia `*aPtr`.

Con l'**allocazione statica** le si collega a mano: `a = 10; aPtr = &a;` e ora `*aPtr` vale 10.



allocazione statica

L'**allocazione dinamica** invece chiede memoria nuova sull'heap mentre il programma gira, con le funzioni della libreria `stdlib.h`:

```
int *aPtr;
aPtr = (int*) malloc(sizeof(int));
```

`aPtr` ora punta a un `int` sull'heap; `*aPtr` resta **non definito** finché non ci scrivi.

```
int *aPtr;
aPtr = (int*) calloc(1, sizeof(int));
```

Come sopra, ma la memoria è **inizializzata a 0**: `*aPtr` vale 0.

Funzione	Cosa fa
`void *malloc(size_t n);`	alloca `n` **byte** e restituisce l'indirizzo dove sono
`void *calloc(size_t n, size_t size);`	alloca un array di `n` oggetti grandi `size`, mette **0** in ogni posizione e restituisce l'indirizzo del primo
`void free(void *p);`	libera la memoria allocata con `malloc` o `calloc`

Entrambe restituiscono:
- un puntatore `void*`, che va convertito con un casting al tipo giusto (`(int*)`);
- `NULL` se non riescono ad allocare: **bisogna sempre controllare** che il risultato sia `!= NULL` prima di usarlo.

`size_t` è il tipo intero senza segno restituito da `sizeof`.

> **Definizione — `void*`.** Un `void*` è un **puntatore generico**: un indirizzo senza un tipo. Si usa anche nei prototipi di funzioni che devono accettare puntatori di qualsiasi tipo. Siccome non dice cosa c'è all'indirizzo, **non si può dereferenziare**: prima si fa il casting a un tipo concreto.

```
int a = 10;
void *aP = &a;
printf("%d\n", *aP);          // errore
```

```
int a = 10;
void *aP = &a;
printf("%d\n", *(int*)aP);    // 10
```

La memoria dinamica resta occupata finché non la si libera con `free(aPtr);`, da fare quando l'oggetto non serve più. Gli errori comuni:
- **memory leak**: non liberare la memoria e poi perdere il puntatore. Quella memoria non si può più né usare né liberare, e se succede in un ciclo il programma può esaurire la memoria;
- liberare memoria allocata in modo statico (`free(&a)`);
- usare un puntatore dopo averlo liberato.

Vantaggi dell'allocazione dinamica	Svantaggi
più controllo sulla memoria usata, meno spreco, programmi più generici (la dimensione si decide mentre gira), più modularità e passaggio di parametri più facile	la gestione della memoria è in più, a carico tuo; rischio di memory leak

3.4 Array di puntatori e matrici dinamiche

Una matrice `int b[N][M]` ha due limiti: `N` e `M` si decidono quando si scrive il programma, e una funzione che la riceve deve conoscere `M` (`int b[][M]`, Sezione 3.2). Se le dimensioni si leggono da input servono una matrice costruita sull'heap e, per farla, un **array di puntatori**.

I puntatori sono variabili, quindi se ne possono fare array:
- `int *ps[10];` alloca 10 puntatori **non inizializzati**: bisogna assegnare un valore a ognuno;

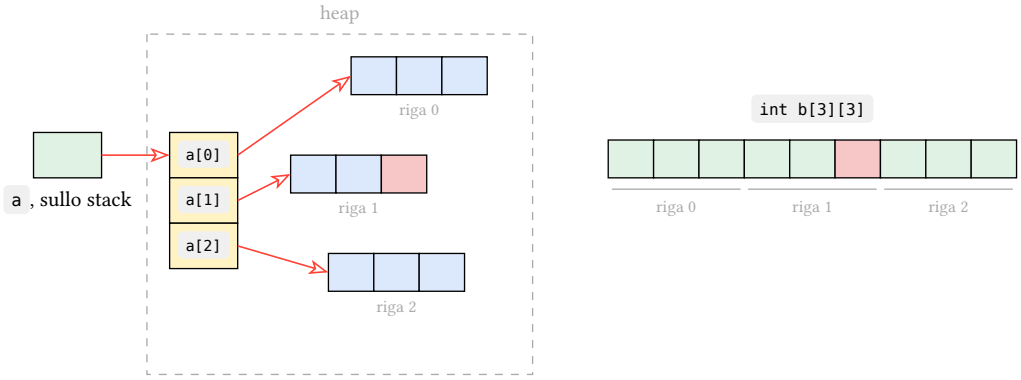
- `int **ps = (int**) malloc(n * sizeof(int*));` alloca sull'heap `n` puntatori a `int`, non inizializzati; con `calloc(n, sizeof(int*))` partono tutti a `NULL`.

Il tipo `int **` si legge «puntatore a puntatore a `int`». Viene dalla regola di sempre: `malloc` restituisce l'indirizzo del primo elemento, quindi per elementi `int` serve un `int *`, per elementi `int *` serve un `int **`.

Ora si fa puntare ogni `ps[i]` a un array di `int` allocato a parte: ogni array è una riga. È la **matrice dinamica allocata per righe**:

```
int **a = (int**) malloc(3 * sizeof(int*));   // 3 puntatori, uno per riga
for (int i = 0; i < 3; i++)
    a[i] = (int*) malloc(3 * sizeof(int));    // la riga i: 3 int
```

Nel disegno, a sinistra questa matrice, a destra `int b[3][3]` per confronto. In giallo i puntatori, in blu e verde gli `int`; in rosso lo stesso elemento `[1][2]` nelle due matrici.



`a[1][2]` si legge in due passi, come `(a[1])[2]`:
1. `a[1]` è il secondo puntatore giallo: l'indirizzo della riga 1;
2. `[2]` su quell'indirizzo dà il terzo `int` della riga.

Con `b[1][2]` invece c'è un blocco unico: si salta una riga intera da 3 `int`, poi altri 2 `int`. Per questo il compilatore deve conoscere la lunghezza della riga, mentre con `a` gli indirizzi delle righe sono scritti nei puntatori gialli. Le differenze:

	`int b[N][M]`	**`int **a` allocata per righe**
dimensioni	fisse, scritte nel programma	decise mentre il programma gira
in memoria	un blocco unico, righe una dopo l'altra	righe sparse sull'heap
righe	tutte lunghe `M`	ognuna può avere una lunghezza diversa, senza sprechi
parametro	`int b[][M]`: serve `M`	`int **a`: `M` non serve
liberare	non serve	`free` di ogni riga, poi di `a`

L'ordine del `free` conta: prima le righe, poi l'array dei puntatori. Al contrario, dopo `free(a)` gli indirizzi delle righe non si possono più leggere, e le righe restano occupate per sempre (memory leak).

```
for (int i = 0; i < 3; i++)
    free(a[i]);
free(a);
```

Un esempio del prof mette tutto insieme. Leggere `M`, `N` (con `M ≥ N ≥ 0`) e una matrice `M × M` allocata per righe; stampare la somma della sottomatrice `N × N` in alto a sinistra, con una funzione `int sum_submatrix(int **matrix, int n)`. Con `M = 4`, `N = 2` e righe `(1,2,3,4)`, `(5,6,7,8)`, `(9,10,11,12)`, `(13,14,15,16)` stampa `1 + 2 + 5 + 6 = 14`.

```c
#include <stdio.h>
#include <stdlib.h>

int sum_submatrix(int **matrix, int n) {
    int i, j, sum = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            sum += matrix[i][j];
    return sum;
}

int main(void) {
    int m, n, i, j;
    int **A;
    scanf("%d%d", &m, &n);
    if (m < 0 || n < 0 || n > m)
        return 1;
    if (m == 0) {
        printf("0\n");
        return 0;
    }
    A = malloc(m * sizeof(int *));
    if (A == NULL) return 1;
    for (i = 0; i < m; i++) {
        A[i] = malloc(m * sizeof(int));
        if (A[i] == NULL) {
            while (i > 0) free(A[--i]);
            free(A);
            return 1;
        }
    }
    for (i = 0; i < m; i++)
        for (j = 0; j < m; j++)
            if (scanf("%d", &A[i][j]) != 1) {
                for (i = 0; i < m; i++)
                    free(A[i]);
                free(A);
                return 1;
            }
    printf("%d\n", sum_submatrix(A, n));
    for (i = 0; i < m; i++) free(A[i]);
    free(A);
    return 0;
}
```

Le righe nuove:

- `malloc` è senza `(int**)`: in C un `void*` si converte da solo in qualunque puntatore, quindi il casting si può omettere;
- se fallisce la `malloc` della riga `i`, le righe da 0 a `i-1` sono già allocate e vanno liberate prima di uscire. `free(A[--i])` decrementa `i` **prima** di usarlo, quindi libera `A[i-1]`, …, `A[0]`; poi si libera `A`;
- `scanf` restituisce quanti valori ha letto: se non è 1 l'input è sbagliato, e prima di uscire si libera tutto.

> **Esercizio — diagonale**
> Leggere `M` e una matrice quadrata `M × M`, allocata dinamicamente per righe. Con una funzione, cambiare il segno degli elementi sulla diagonale principale e stampare la matrice, una riga per riga di output. `(1,2,3), (4,5,6), (7,8,9)` diventa `(-1,2,3), (4,-5,6), (7,8,-9)`.

3.5 Stringhe

In C non c'è un tipo stringa come in JavaScript: una **stringa è un array di `char` terminato da `'\0'`**, il carattere nullo (valore 0). Ogni funzione sulle stringhe va avanti finché non trova `'\0'`: è l'unico modo per sapere dove finiscono. Ci sono due modi di crearne una, e **non sono la stessa cosa**:

```c
char s1[] = "ciao";
s1[2] = 'A';
printf("s1=%s\n", s1);   // s1=ciAo
```

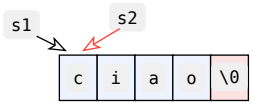
Un **array** di 5 `char` sullo stack, inizializzato con una copia di `"ciao"`: si può modificare.

```c
char *s1 = "ciao";
s1[2] = 'A';             // crash
printf("s1=%s\n", s1);
```

Un **puntatore** a una stringa costante, che sta in una zona di sola lettura: scriverci fa terminare il programma.

Attenzione a copiare: l'assegnamento fra puntatori copia **l'indirizzo**, non i caratteri, e da lì in poi i due nomi modificano lo stesso array. Per una copia vera si copiano i caratteri uno a uno, **terminatore compreso**:

```c
char s1[] = "ciao";
char *s2 = s1;     // stesso array
s2[2] = 'A';
// s1=ciAo, s2=ciAo
```




```c
char s1[] = "ciao";
char s2[5];
int i = 0;
while (s1[i] != '\0') {
    s2[i] = s1[i];
    i++;
}
s2[i] = '\0';      // non dimenticarlo
s2[2] = 'A';       // s1=ciao, s2=ciAo
```

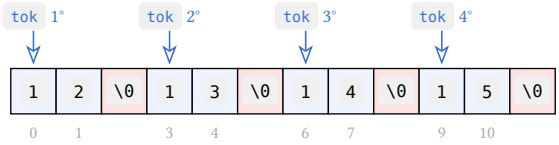
Senza il `'\0'` finale, `printf("%s", s2)` continuerebbe a leggere la memoria dopo l'array. Le funzioni già pronte stanno in `string.h`:

Funzione	Cosa fa
`strlen(str)`	lunghezza, **senza** il terminatore: `strlen("ciao")` = 4 (`sizeof` dell'array `"ciao"` = 5)
`strcpy(dest, src)`	copia `src` in `dest` e restituisce `dest`. `dest` deve essere abbastanza grande
`strcmp(s1, s2)`	confronto lessicografico: **negativo** se `s1` viene prima, **0** se uguali, **positivo** se dopo. Non si usa `==`, che confronterebbe gli indirizzi
`strcat(dest, src)`	aggiunge una copia di `src` in fondo a `dest`
`strncpy`, `strncmp`, `strncat`	come sopra, ma solo sui primi `n` caratteri
`strtok(str, delim)`	divide la stringa in **token** separati dai caratteri di `delim`

`strtok` si usa in due tempi: la prima chiamata riceve la stringa, le successive `NULL` (vuol dire «continua da dove eri rimasto»). Restituisce un puntatore al token successivo, o `NULL` quando sono finiti. Per farlo **modifica la stringa originale**: sostituisce ogni separatore con `'\0'`, e `tok` punta dentro la stringa stessa.

```c
char s[100], *tok;
scanf("%[^\n]", s);          // 12,13,14,15
tok = strtok(s, ",");
printf("Token:%s\n", tok);
while ((tok = strtok(NULL, ",")) != NULL)
    printf("Token:%s\n", tok);
printf("Stringa:%s\n", s);
```

```
Token:12
Token:13
Token:14
Token:15
Stringa:12
```



dopo `strtok`: le virgole sono diventate `'\0'`, quindi `s` ora è solo `"12"`

Per questo `Stringa:12` alla fine, e per questo `strtok` su `char *s = "12,13,14,15"` (sola lettura) dà `Segmentation fault`.

Per i singoli caratteri c'è `ctype.h`. Le funzioni di controllo restituiscono un valore **diverso da zero** se la condizione è vera:

Funzione	**Vero se `c` è…**
`isalpha(c)`, `isupper(c)`, `islower(c)`	una lettera, maiuscola, minuscola
`isdigit(c)`, `isalnum(c)`, `isspace(c)`	una cifra, cifra o lettera, uno spazio bianco
`toupper(c)`, `tolower(c)`	convertono il carattere se possono, altrimenti lo restituiscono uguale: `toupper('7')` = `'7'`

Per leggere caratteri e stringhe, e per convertire una stringa in numero:

Funzione	**Cosa fa**
`scanf("%c", &c)`	un carattere
`scanf("%s", s)`	una parola: si ferma al primo spazio. Niente `&`: `s` è già un indirizzo
`scanf("%[^\n]", s)`	tutta la riga fino all'a capo, spazi compresi
`getchar()`	un carattere da stdin
`fgets(str, num, stream)`	al più `num-1` caratteri da `stream` (per la tastiera `stdin`), poi aggiunge `'\0'`
`sscanf(s, format, ...)`	come `scanf`, ma legge dalla stringa `s` invece che da stdin
`atoi(str)`, `atol(str)`, `atof(str)`	stringa ⟶ `int`, `long`, `double` (in `stdlib.h`)

Con `scanf("%s", s)` nessuno controlla la lunghezza: se la parola è più lunga dell'array, si scrive oltre la fine. Nelle soluzioni del prof si limita con la larghezza: `scanf("%1000s", buffer)` con `char buffer[1001]` (uno in più per il `'\0'`).

Esercizio — concatenazione
Leggere lunghezza e contenuto di due stringhe (al più 1000 caratteri), allocando per ciascuna esattamente lo spazio indicato più il terminatore. Scrivere `char* my_strcat1(char *s1, char *s2)`, che alloca una **nuova** stringa con la concatenazione, senza usare `strcat` o `strncat`, e stamparla. Lunghezze 10 e 3, stringhe `ciaoatutti` e `voi` ⟶ `ciaoatuttivoi`.

Esercizio — anagrammi
Scrivere `int anagramma(char *s1, char *s2)`, che restituisce 1 se le due stringhe sono anagrammi e 0 altrimenti, e un programma che la usa. Suggerimento: contare in array di 256 elementi le occorrenze di ogni carattere (indice `(unsigned char) s[i]`). `pizza` e `pazzi` sono anagrammi.

Esercizio — prefisso comune
`LCP(s1, s2)` è la lunghezza del più lungo prefisso comune a `s1` e `s2`. Leggere `n` e poi `n` stringhe di al più 500 caratteri; per ogni `0 < i < n` stampare `LCP(S[i-1], S[i])`. `LCP(AABB, AAZZ) = 2`, `LCP(mississippi, missi) = 5`, `LCP(laboratorio, laboratoriale) = 10`.

3.6 Puntatori a funzioni

In JavaScript una funzione è un valore e si passa come qualsiasi altro. In C le funzioni **non sono variabili**, ma se ne può prendere un **puntatore**, da passare come parametro o da usare per chiamare la funzione:

```c
int sum(int x, int y) { return x + y; }
int mul(int x, int y) { return x * y; }

int main(void) {
    int (*f)(int, int);
    f = sum;
    printf("%d\n", (*f)(3, 6));     // 9
    f = mul;
    printf("%d\n", (*f)(3, 6));     // 18

    int (*g[2])(int, int) = {sum, mul};
    printf("%d\n", (*g[1])(3, 6));  // 18
}
```

`int (*f)(int, int)` si legge: `f` è un puntatore a una funzione che prende due `int` e restituisce un `int`. Le parentesi attorno a `*f` servono: `int *f(int, int)` sarebbe invece il prototipo di una funzione che restituisce un `int *`.

Il nome di una funzione, da solo, è il suo indirizzo: `f = sum`. `g` è un **array di puntatori a funzioni**.

L'uso tipico è `qsort` di `stdlib.h`, che ordina un array di **qualsiasi tipo**. Per essere generica usa `void*` e si fa passare la funzione di confronto:

```c
void qsort(void *base, size_t num, size_t size,
           int (*compar)(const void *, const void *));
```

- `base` è l'array, `num` il numero di elementi, `size` la dimensione di un elemento: `qsort` non conosce il tipo, quindi sposta blocchi di `size` byte;
- `compar` riceve gli **indirizzi** di due elementi e restituisce, come `strcmp`, un numero negativo, zero o positivo.

```c
int compInt(const void *a, const void *b) {
    int *ac = (int *) a;
    int *bc = (int *) b;
    return *ac - *bc;
}

int a[5] = {17, 9, 2, 43, 0};
qsort(a, 5, sizeof(int), compInt);
// 0 2 9 17 43
```

È lo stesso schema di `array.sort((a, b) => a - b)` in JavaScript. La differenza è che `compInt` riceve dei `void*`, e deve prima convertirli al tipo vero per poterli dereferenziare. `const` dice che la funzione non modificherà gli elementi.

> **Esercizio — suffissi ordinati**
> Leggere una stringa di al più 1000 caratteri e stampare tutti i suoi suffissi, uno per riga, in ordine lessicografico. Si può usare `qsort`. Da `ciao`: `ao`, `ciao`, `iao`, `o`.

3.7 Trovare gli errori

Un programma con i puntatori spesso compila senza errori, ma mentre gira dà risultati sbagliati o termina con `Segmentation fault` (Sezione 3.1). Per capire **dove** sbaglia ci sono strumenti via via più potenti: le stampe, `assert`, e due programmi fatti apposta, gdb e valgrind.

Il modo più semplice è **stampare i valori delle variabili**. Ma `printf` scrive su `stdout`, che è **bufferizzato**: il testo si può accumulare in un buffer (una zona di memoria) invece di comparire subito a video, e se il programma termina per un errore quello che c'è nel buffer si perde. Su `stderr` (Sezione 1.3) invece

il testo esce subito. Per scriverci si usa `fprintf`, che è come `printf` ma riceve come primo argomento il flusso:

```
fprintf(stderr, "Messaggio\n");
```

> **Trappola**: se dopo `printf("prima");` il programma va in `Segmentation fault`, su terminale `prima` non compare, e sembra che il programma si sia fermato **prima** della `printf`. Con `fprintf(stderr, "prima");` compare. Su terminale `stdout` svuota il buffer a ogni `\n`: si perdono le stampe senza `\n`.

Una stampa però mostra solo i valori che si decide di guardare. Con la macro `assert` (da `assert.h`) si scrive invece una condizione che in quel punto **deve** essere vera. `assert` riceve un'espressione intera: se è vera l'esecuzione continua, se è falsa stampa un errore su `stderr` e **termina subito** il programma. Qui l'utente deve inserire un numero positivo, e sotto c'è cosa succede se inserisce `-3`:

```
#include <stdio.h>
#include <assert.h>

int main(void) {
    int x;
    scanf("%d", &x);
    assert(x > 0);
    printf("Hai inserito %d\n", x);
}
```

```
$ ./main
-3
main: main.c:7: main: Assertion `x > 0' failed.
Aborted (core dumped)
```

Il messaggio dice il programma, il file, la riga, la funzione e la condizione falsa; `Aborted` è la terminazione immediata. Gli `assert` servono durante lo sviluppo. Per spegnerli tutti senza cancellarli si definisce la macro `NDEBUG` **prima** di includere `assert.h`, perché è al momento dell'`include` che `assert` decide se fare il controllo:

```
#define NDEBUG
#include <stdio.h>
#include <assert.h>
```

Ora con `-3` il programma stampa `Hai inserito -3`.

Aggiungere e togliere stampe è lento, e su un programma grande non basta. Ci sono due programmi fatti apposta:

- **gdb** esegue il programma passo per passo e fa vedere il valore delle variabili;
- **valgrind** sorveglia gli accessi alla memoria e segnala quelli sbagliati.

Nessuno dei due entra nel codice di una libreria già compilata: le funzioni vanno provate **prima** di metterle in una libreria.

Per entrambi si compila con l'opzione `-g`, che mette nell'eseguibile i nomi delle variabili e i numeri di riga del sorgente: così gdb e valgrind parlano di `x` e di `main.c:6` invece che di indirizzi (`-Wall` e `-pedantic` sono in Sezione 2.1). Poi si avvia gdb sull'eseguibile:

```
$ gcc -g -Wall -pedantic main.c -o main
$ gdb main                  # in generale: gdb [opzioni] eseguibile
```

gdb apre un suo prompt, `(gdb)`, dove si scrivono i comandi: `run` avvia il programma, `quit` chiude gdb. Lanciato così, però, il programma corre fino in fondo. Per guardarlo da dentro bisogna dire a gdb **dove fermarsi**:

- un **breakpoint** è un punto del codice: quando l'esecuzione ci arriva, si ferma;
- un **watchpoint** è una variabile da sorvegliare: l'esecuzione si ferma quando il suo valore cambia. Funziona solo finché la variabile è nel suo scope (per una variabile locale, finché si è dentro la sua funzione).

Comando	Dove si ferma
`b 6`	breakpoint alla riga 6
`b main.c:6`	breakpoint alla riga 6 del file `main.c`
`b push`	breakpoint sulla prima istruzione della funzione `push`
`watch a`	watchpoint sulla variabile `a`
`delete n`, `enable n`, `disable n`	toglie, riattiva, disattiva il breakpoint o watchpoint numero `n` (gdb li numera man mano che li crei)

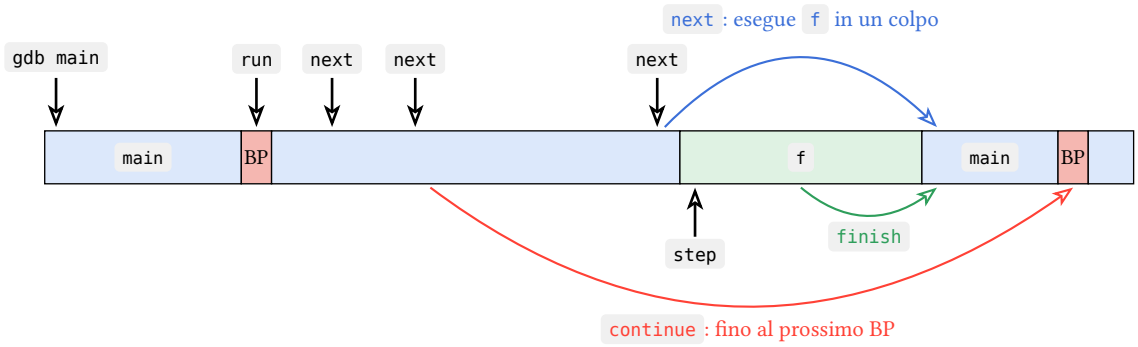
Fermi su un breakpoint, si guarda lo **stato** del programma:

Comando	Cosa stampa
`p x`, `p &x`	il valore di `x`, il suo indirizzo (`p` sta per *print*)
`p a[3]@5`	5 elementi dell'array `a` a partire da `a[3]`
`bt`	*backtrace*: la pila dei record di attivazione (Sezione 3.3), cioè quale funzione ha chiamato quale fino a quella in cui si è fermi
`x` *indirizzo*	il contenuto della memoria a quell'indirizzo

Poi si fa **ripartire** l'esecuzione:

Comando	Fin dove va avanti
`continue`	fino al prossimo breakpoint
`next`	un'istruzione, **senza entrare** nelle funzioni chiamate: una chiamata viene eseguita tutta in un colpo
`step`	un'istruzione, **entrando** nella funzione se l'istruzione è una chiamata
`finish`	fino all'uscita dalla funzione in cui si è

Il disegno mette insieme i comandi. La barra è l'esecuzione del programma nel tempo, da sinistra a destra: in azzurro il codice di `main`, in verde una funzione `f` chiamata da `main`, in rosso i due breakpoint (BP). Le frecce dritte indicano dove si ferma il programma dopo ogni comando; quelle curve, da dove a dove salta un comando.



valgrind esegue il programma tenendo traccia di **tutti gli accessi alla memoria**, e stampa un messaggio quando la memoria è usata male. Trova per esempio:

- variabili usate prima di essere inizializzate;
- **stack overflow**: lo stack si riempie, per esempio con una ricorsione che non si ferma mai;
- memory leak (Sezione 3.3).

Anche qui si compila con `-g`, poi si fa partire il programma **dentro** valgrind:

```
$ gcc -g -Wall -pedantic main.c -o main
$ valgrind ./main [argomenti]         # argomenti di main, se ne ha
$ valgrind --leak-check=full ./main   # in più, i dettagli dei memory leak
```

4 Strutture dati

Cosa devi saper fare

- allocare una `struct` sull'heap e usarne i membri con `->`;
- costruire, scorrere e liberare una lista concatenata, inserendo in testa e in coda;
- inserire in un albero binario di ricerca, visitarlo in ordine e liberarlo, con funzioni ricorsive.

In JavaScript un array cresce da solo con `push`, e la memoria la libera il garbage collector. In C un array ha la dimensione decisa quando lo si crea. Una struttura che cresce mentre il programma gira si costruisce a mano: ogni pezzo è una `struct` allocata sull'heap, e i pezzi sono collegati da puntatori. Il primo passo è quindi allocare una `struct` con `malloc`.

4.1 Puntatori a struct

Una `struct` si alloca sull'heap come un `int` (Sezione 3.3): `malloc(sizeof(tipo))` riserva i byte e restituisce un `void*`, che si converte con un casting. L'esempio del prof è una `struct` che chiama `Stack` (in inglese «pila»), fatta da un array di `N` interi e da un intero `n`:

```c
#define N 100

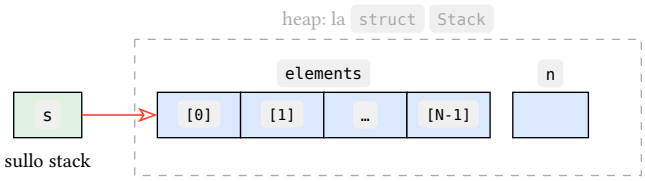
typedef struct {
    int elements[N];
    int n;
} Stack;

Stack *s = (Stack*) malloc(sizeof(Stack));
s->n = 0;
/* ... */
free(s);
```

La riga con `malloc`, pezzo per pezzo:

- `sizeof(Stack)` è la dimensione di tutta la `struct`: l'array, l'intero e l'eventuale padding. Qui 404 byte;
- `(Stack*)` converte il `void*` di `malloc` in un puntatore a `Stack` (si può omettere, come in Sezione 3.4);
- `s` è un **puntatore a `Stack`**: è una variabile locale, quindi sta sullo stack, e contiene solo un indirizzo. La `struct` vera sta sull'heap.

Nel disegno, in verde ciò che sta sullo stack, nel riquadro tratteggiato l'heap, in blu i membri della `struct`.



> `Stack` qui è solo un nome: questo `s` non c'entra con lo stack della memoria. La `struct` puntata sta sull'heap.

Per usare un membro bisogna prima seguire il puntatore, poi scegliere il membro. Il C ha un operatore apposta, `->`:

con `->`	equivale a	significato
`s->n`	`(*s).n`	il membro `n` della `struct` puntata da `s`

Regola: `.` si usa su una `struct`, `->` su un **puntatore** a `struct`.

> **`*s.n` non funziona.** `.` lega più di `*`, quindi si legge `*(s.n)`; ma `s` non è una `struct` e il compilatore si ferma: `'s' is a pointer; did you mean to use '->'?`. Per questo nella forma lunga servono le parentesi, `(*s).n`.

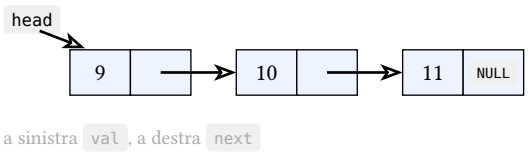
`->` serve anche con un array di puntatori a `struct` (Sezione 3.4). Se `studenti` è uno `Studente **`, `studenti[i]` è un puntatore a `Studente` e `studenti[i]->matricola` è la sua matricola. Per leggerla con `scanf` serve il suo indirizzo: `&studenti[i]->matricola` vale `&(studenti[i]->matricola)`, perché `->` lega più di `&`.

> **Esercizio — ordinare con merge sort**
> Allocare dinamicamente un array di puntatori a `struct Studente` (matricola ed età), leggere numero e dati degli studenti, ordinare l'array per matricola con Merge Sort e stamparlo, poi liberare tutta la memoria. Si ordinano i **puntatori**: `array[k] = L[i]` sposta un indirizzo, non la `struct`.

4.2 Liste concatenate

Una `struct` può contenere un puntatore a una `struct` del suo stesso tipo: si chiama **self-referential struct**. Su questo si basa la **lista concatenata**: ogni elemento (**nodo**) contiene un valore e l'indirizzo del nodo **successivo**. Della lista si tiene solo il puntatore al primo nodo, la **testa** (`head`). L'ultimo nodo ha `next = NULL`, che segna la fine.

```c
struct n {
    int val;
    struct n *next;
};
typedef struct n Node;
```



a sinistra `val`, a destra `next`

Nel disegno ogni nodo è un rettangolo diviso in due, e le frecce sono i puntatori. Nella `struct` va scritto `struct n *next` e non `Node *next`: il nome `Node` nasce solo con il `typedef`, una riga dopo.

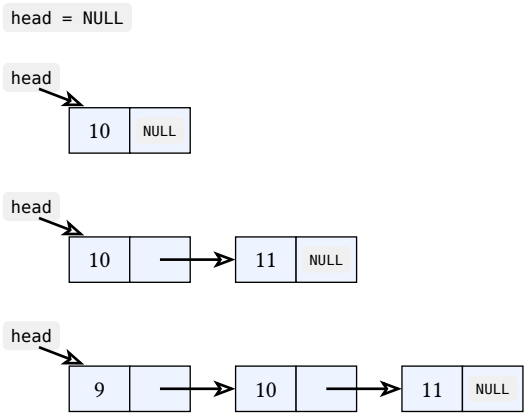
L'esempio del prof costruisce questa lista con tre nodi locali, mettendo il 10, poi l'11 in coda e il 9 in testa. A destra la lista dopo ogni passo:

```c
Node *head = NULL;      // lista vuota

Node elem;
elem.val = 10;
elem.next = NULL;
head = &elem;           // un nodo

Node elem1;
elem1.val = 11;
elem1.next = NULL;
elem.next = &elem1;     // in coda

Node elem2;
elem2.val = 9;
elem2.next = head;
head = &elem2;          // in testa
```



- **in coda**: l'ultimo nodo, che puntava a `NULL`, ora punta al nuovo;
- **in testa**: il nuovo nodo punta alla vecchia testa, poi diventa lui la testa. L'ordine conta: con `head = &elem2` per primo, la vecchia testa andrebbe persa.

Per scorrere la lista un puntatore `l` parte dalla testa e segue i `next` fino a `NULL`. È un `for` su un array in cui, al posto dell'indice, c'è un puntatore:

```
Node *l = head;
while (l != NULL) {
    printf("%d ", l->val);
    l = l->next;
}
// stampa: 9 10 11
```

Questi nodi però sono variabili locali: stanno sullo stack e spariscono quando la funzione finisce, quindi la lista non sopravvive alla funzione che la crea. Una lista vera alloca ogni nodo sull'heap con `malloc`, come la `struct` della sezione prima (Sezione 4.1). Così fa la soluzione del prof all'esercizio sulla lista di studenti, dove ogni nodo è uno studente e i nuovi si aggiungono in coda:

```
typedef struct Studente {
    int matricola;
    int eta;
    struct Studente *next;
} Studente;

Studente* aggiungiStudente(Studente *head, int matricola, int eta) {
    Studente *nuovo = (Studente *) malloc(sizeof(Studente));
    if (nuovo == NULL) return head;
    nuovo->matricola = matricola;
    nuovo->eta = eta;
    nuovo->next = NULL;
    if (head == NULL) return nuovo;
    Studente *temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = nuovo;
    return head;
}
```

1. Alloca il nuovo nodo e ne riempie i membri. `next = NULL` perché sarà l'ultimo.
2. Se la lista è vuota (`head == NULL`), il nuovo nodo è tutta la lista: lo restituisce come testa.
3. Altrimenti `temp` parte dalla testa e avanza finché `temp->next` non è `NULL`, cioè fino all'ultimo nodo, e lì attacca il nuovo.

La funzione **restituisce la testa**, e chi la chiama deve assegnarla: `head = aggiungiStudente(head, 1, 23);`. Serve perché al passo 2 la testa cambia, e la funzione ha solo una copia di `head` (passaggio per valore, Sezione 2.6).

Alla fine ogni nodo va liberato con `free`:

```
void liberaLista(Studente *head) {
    Studente *temp;
    while (head != NULL) {
        temp = head;
        head = head->next;    // prima avanzo...
        free(temp);           // ...poi libero
    }
}
```

L'ordine conta: dopo `free(temp)` leggere `temp->next` sarebbe un uso dopo `free`. Per questo si avanza `head` prima di liberare.

Al posto del valore restituito si può passare l'**indirizzo** della testa, come in `swap` (Sezione 3.1). Il prof lo fa per la funzione di test, chiamata dal `main`:

```c
void testLista(Studente **lista) {
    *lista = aggiungiStudente(*lista, 1, 23);
    *lista = aggiungiStudente(*lista, 2, 21);
    /* ... stampa, ordina, estrae ... */
    liberaLista(*lista);
}

int main(void) {
    Studente *lista = NULL;
    testLista(&lista);
    return 0;
}
```

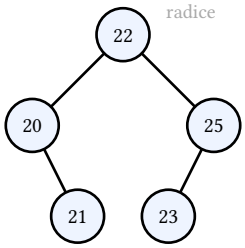
`lista` è uno `Studente *`, quindi il suo indirizzo `&lista` è uno `Studente **`. Dentro `testLista`, `*lista` è proprio la variabile `lista` del `main`: assegnarle una nuova testa cambia quella.

> **Esercizio — lista di studenti**
> Completare il programma: `stampaLista`; `ordinaPerEta`, che ordina la lista per età crescente con un bubble sort che scambia i **valori** dei nodi (non i puntatori); `estraiPerEta`, che restituisce una **nuova** lista con gli studenti dell'età letta da tastiera. Alla fine liberare entrambe le liste.

4.3 Alberi binari di ricerca

La lista ha un puntatore per nodo. Con **due** puntatori, `left` e `right`, si ottiene un **albero binario**: ogni nodo ha al massimo due figli, uno a sinistra e uno a destra, e il nodo in cima si chiama **radice**. In un **albero binario di ricerca** i nodi sono in ordine: a sinistra di un nodo ci sono solo valori minori, a destra solo valori maggiori o uguali.

Nell'esercizio del prof i nodi sono studenti ordinati per età: i più giovani a sinistra, i più anziani a destra. Nel disegno ogni cerchio è un nodo con la sua età, e gli studenti sono stati inseriti nell'ordine 22, 20, 25, 21, 23:



Il nodo ha gli stessi membri della lista, ma con `left` e `right` al posto di `next`. `creaNodo` lo alloca e mette i due figli a `NULL`; se `malloc` fallisce, `exit(1)` termina il programma.

```c
typedef struct Studente {
    int matricola;
    int eta;
    struct Studente *left;
    struct Studente *right;
} Studente;

Studente* creaNodo(int matricola, int eta) {
    Studente *nuovo = (Studente *) malloc(sizeof(Studente));
    if (nuovo == NULL) exit(1);
    nuovo->matricola = matricola;
    nuovo->eta = eta;
    nuovo->left = NULL;
    nuovo->right = NULL;
    return nuovo;
}
```

Per inserire si scende dalla radice: a sinistra se l'età è minore, a destra altrimenti, finché si trova un posto vuoto. La funzione è **ricorsiva**: ogni chiamata guarda un nodo e passa il lavoro al figlio giusto.

```c
Studente* inserisciStudente(Studente *radice, int matricola, int eta) {
    if (radice == NULL)                  // posto vuoto: il nodo va qui
        return creaNodo(matricola, eta);
    if (eta < radice->eta)
        radice->left = inserisciStudente(radice->left, matricola, eta);
    else
        radice->right = inserisciStudente(radice->right, matricola, eta);
    return radice;
}
```

Nel disegno, l'inserimento di 21:

1. $21 < 22$: si scende a sinistra, nel nodo 20;
2. $21 \geq 20$: si scende a destra, dove c'è `NULL`;
3. posto vuoto: `creaNodo` crea il nodo, e la chiamata sul nodo 20 lo attacca come suo `right`.

Come per la lista, la funzione restituisce la radice, che cambia solo quando l'albero è vuoto. Si chiama quindi `radice = inserisciStudente(radice, m, e);`.

Anche stampa e liberazione sono ricorsive:

```c
void stampaInOrder(Studente *radice) {
    if (radice == NULL) return;
    stampaInOrder(radice->left);
    printf("%d %d\n", radice->matricola,
                      radice->eta);
    stampaInOrder(radice->right);
}
```

```c
void liberaAlbero(Studente *radice) {
    if (radice == NULL) return;
    liberaAlbero(radice->left);
    liberaAlbero(radice->right);
    free(radice);
}
```

La visita **in ordine** stampa prima tutto il sottoalbero sinistro, poi il nodo, poi il destro. Siccome a sinistra stanno i più giovani, le età escono in ordine crescente: 20, 21, 22, 23, 25. In `liberaAlbero` invece il nodo si libera **per ultimo**: liberato prima, i suoi `left` e `right` non si potrebbero più leggere.

Esercizio — albero di studenti
Leggere coppie matricola/età e inserirle nell'albero, fermandosi quando la matricola è 0. Stampare gli studenti in ordine di età, poi quelli di un'età letta da tastiera (`stampaEta` , ricorsiva), e liberare la memoria.

Riepilogo

Errore	Quando	Esempio
di compilazione	sintassi, tipi incompatibili, nome non dichiarato	`expected ';'`, `'countA' undeclared`, `implicit declaration of function`, `makes pointer from integer without a cast`
del linker	manca la **definizione** di qualcosa di dichiarato	`undefined reference to 'max'` (anche se è `static` in un altro file)
di esecuzione	compila ma si comporta male	`Segmentation fault`: `NULL`, memoria non allocata, scrivere su `char *s = "…"`

Scrittura	Significato
`int *p;`	puntatore a `int`, valore non definito
`p = &x;` / `*p = 5;`	`p` punta a `x` / scrive 5 in `x`
`a + i`, `*(a + i)`	`&a[i]`, `a[i]` (avanza di `i` **elementi**)
`int **m`	puntatore a puntatore: array di righe, o indirizzo di un puntatore da modificare
`s->membro`	`(*s).membro`
`void *`	puntatore generico: casting prima di dereferenziare
`int (*f)(int, int)`	puntatore a funzione

Variabile	Dove	Visibile / durata
locale	stack	nel blocco / fino alla `}`
locale `static`	dati statici	nel blocco / tutto il programma, mantiene il valore
globale	dati statici	nel file, e negli altri con `extern` / tutto il programma
globale `static`	dati statici	solo nel suo file / tutto il programma
`malloc`, `calloc`	heap	tramite puntatore / fino a `free`

Trappole

- `5 / 2` vale 2; `(float) a / b` divide fra reali, `(float) (a / b)` no.
- `2 < a < 5` è sempre vero; `switch` senza `break` prosegue nel `case` dopo.
- `scanf` vuole gli indirizzi (`&x`, ma non per un array di `char`) e `%lf` per i `double`.
- Una variabile in un blocco interno copre quella esterna, e la condizione del `do-while` vede quella esterna.
- `a + i` e non `a + i * sizeof(int)`; `sizeof` di un array ≠ `sizeof` di un puntatore.
- `char s[] = "ciao"` si modifica, `char *s = "ciao"` no; `s2 = s1` copia l'indirizzo; le stringhe si confrontano con `strcmp`, non con `==`.
- `strtok` modifica la stringa; `strlen` non conta `'\0'`, ma lo spazio da allocare sì (`n + 1`).
- Controllare `!= NULL` dopo `malloc`; liberare le righe prima dell'array di righe, il nodo dopo aver letto `next`.